

Lesson 14: Provider Essentials

The idea behind Provider

In Lesson 13 you saw the pain of passing data down by hand. Provider fixes it: you place your data in one spot near the top of the app, and *any* widget below can read it -- no passing through middlemen.

Provider is a package (extra code you add to your project). It is the simplest state management tool in Flutter, and the Flutter team itself suggests it for beginners.

ChangeNotifier: data that announces changes

First, you put your data in a class that extends `ChangeNotifier`. This is a normal Dart class with one extra power: it can call `notifyListeners()` to tell the UI "I changed, rebuild":

```
class CartModel extends ChangeNotifier {
  int count = 0;

  void add() {
    count++;
    notifyListeners(); // tell every listening widget to rebuild
  }
}
```

That is all. Normal fields, normal methods -- plus `notifyListeners()` at the end of anything that changes the data.

Providing it to the app

Next, you wrap your app with `ChangeNotifierProvider`. This makes one `CartModel` object available to every widget below it:

```
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => CartModel(),
      child: const MyApp(),
    ),
  );
}
```

`create` builds the object once. From here on, any screen in `MyApp` can reach it.

To use the package, add it to your pubspec.yaml:

```
dependencies:
  provider: ^6.1.2
```

Then run flutter pub get.

Reading the data: watch vs read

There are two ways to reach the data, and the difference matters:

```
// 1. WATCH -- rebuilds this widget when the data changes
final cart = context.watch<CartModel>();
Text('Items: ${cart.count}');

// 2. READ -- gets the data once, no rebuild
context.read<CartModel>().add();
```

- Use watch inside build() when the widget *shows* the data. The widget rebuilds on every change.
- Use read inside button callbacks (onPressed) when you only want to *change* the data. No rebuild needed -- the watchers will rebuild on their own.

A common beginner mistake is using watch inside onPressed. It works, but it is wasteful. Remember: watch to show, read to change.

A full tiny example

```
class CartBadge extends StatelessWidget {
  const CartBadge({super.key});

  @override
  Widget build(BuildContext context) {
    final cart = context.watch<CartModel>();
    return Text('Cart: ${cart.count}');
  }
}

class AddButton extends StatelessWidget {
  const AddButton({super.key});

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: () => context.read<CartModel>().add(),
      child: const Text('Add to cart'),
    );
  }
}
```

CartBadge watches and rebuilds on every tap. AddButton only reads. No data passed through constructors, no prop drilling -- both widgets sit anywhere in the tree and just work.

Key terms

- Provider -- a package that shares data with any widget below it, without manual passing.
- ChangeNotifier -- a class that holds data and calls notifyListeners() when the data changes.
- ChangeNotifierProvider -- a widget that creates the data object and shares it with the whole app.
- watch -- read data in build(); the widget rebuilds when the data changes.
- read -- get the data once (often in button callbacks); no rebuild.